



Experiment – 4

Name: Aryan

UID: 24BAI70179

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance: 03/08/2026

Subject: Full Stuck - II

Subject Code: 24CSP-337

EXPERIMENT – 4.1

Aim:

To design and implement an interactive calendar interface for scheduling and managing posts, enabling users to organize, view, and update content efficiently through a user-friendly calendar-based system.

Objectives:

- To understand time-based data visualization in user interface (UI) systems.
- To implement a calendar-based scheduling interface for managing posts.
- To map structured post data onto a temporal calendar layout.
- To enable user interactions such as drag-and-drop for rescheduling posts.
- To improve usability by providing an intuitive visual scheduling experience.

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Frontend: React.js
- Backend: Node.js with Express.js (optional)
- Database: MongoDB / Firebase / MySQL (optional)
- Calendar Library: FullCalendar / React Big Calendar
- Code Editor: Visual Studio Code
- Package Manager: npm
- Browser: Google Chrome / Microsoft Edge

Theory:

A calendar scheduling interface is a visual representation of time that allows users to manage events or posts efficiently. Instead of manually entering dates and times, users interact with a calendar where posts are displayed according to their scheduled time.

The interface maps structured data, such as post title, description, date, and time, onto calendar cells. Modern calendar applications also support interactive features like drag-and-drop, resizing events, and quick editing, improving productivity and user experience.

Using libraries such as Full Calendar simplifies the implementation by providing pre-built views, event handling, and responsive layouts. These features are widely used in content management systems, social media scheduling platforms, project management tools, and event management applications.

Implementation:

1. Set up a React.js project and install the required dependencies.
2. Install and configure a calendar library such as Full Calendar.
3. Design the calendar interface with monthly, weekly, and daily views.
4. Create a data model for scheduled posts containing title, date, time, and status.
5. Display posts dynamically on the calendar.
6. Implement drag-and-drop functionality to allow users to reschedule posts.
7. Enable adding, editing, and deleting scheduled posts.
8. Connect the frontend with a backend/database to store scheduling information.
9. Test user interactions and ensure responsive design across different devices.

Code:

- **App.js**

```
import { useState } from "react";

const WEEKDAYS = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
const MONTH_NAMES = [ "January", "February", "March", "April", "May", "June",
  "July", "August", "September", "October", "November", "December"];

function getMonthGrid(year, month) {
  const firstDay = new Date(year, month, 1).getDay();
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const cells = [];
  for (let i = 0; i < firstDay; i++) cells.push(null);
  for (let d = 1; d <= daysInMonth; d++) cells.push(d);
  while (cells.length % 7 !== 0) cells.push(null);
  return cells;
}
```

```
function dateKey(year, month, day) {  
  return `${year}-${month + 1}-${day}`;  
}  
  
function App() {  
  const today = new Date();  
  
  const [year, setYear] = useState(today.getFullYear());  
  const [month, setMonth] = useState(today.getMonth());  
  const [posts, setPosts] = useState({});  
  const [postText, setPostText] = useState("");  
  const [selectedDay, setSelectedDay] = useState(null);  
  const [draggedFrom, setDraggedFrom] = useState(null);  
  
  const cells = getMonthGrid(year, month);  
  const goPrevMonth = () => {  
    if (month === 0) { setMonth(11); setYear(year - 1);  
    } else {setMonth(month - 1);  
  };  
  const goNextMonth = () => {  
    if (month === 11) {  
      setMonth(0);  
      setYear(year + 1);  
    } else {  
      setMonth(month + 1);  
    }  
  };  
  const addPost = () => {  
    if (!selectedDay || postText.trim() === "") return;  
    const key = dateKey(year, month, selectedDay);  
    const updated = { ...posts };
```

```
};

const handleDrop = (targetKey) => {
  if (!draggedFrom) return;
  if (draggedFrom.key === targetKey) {
    setDraggedFrom(null);
    return;
  }
  const updated = { ...posts };
  const movedPost = updated[draggedFrom.key][draggedFrom.index];
  updated[draggedFrom.key] = updated[draggedFrom.key].filter(
    (_, i) => i !== draggedFrom.index
  );
  updated[targetKey] = [...(updated[targetKey] || []), movedPost];

  setPosts(updated);
  setDraggedFrom(null);
};

return (
  <>
    <div className="coder-header">
      Post Scheduling Calendar
    </div>
    <div className="app-container" style={{
      backgroundColor: "#0a0a0a",
      minHeight: "100vh",
      padding: "40px",
      fontFamily: "'Courier New', Courier, monospace",
```

```
        cursor: day ? "pointer" : "default"

      }}

    >

    {day && (

      <>

      <div style={{ fontWeight: "bold", fontSize: "13px" }}>{day}</div>

      {dayPosts.map((post, i) => (

        <div>

          key={i}

          draggable

          onDragStart={() => handleDragStart(key, i)}

          className="coder-code-area"

          style={{

            backgroundColor: "#003300", color: "#c0ff00",

            fontSize: "11px", borderRadius: "4px", padding: "3px 5px",

            marginTop: "4px", cursor: "grab", display: "flex",

            justifyContent: "space-between",

            alignItems: "center"

          }}

          <span style={{ overflow: "hidden", textOverflow: "ellipsis" }}>{post}</span>

          <span onClick={(e) => { e.stopPropagation(); deletePost(key, i); }} style={{

marginLeft: "4px", cursor: "pointer" }}>X</span>

        </div>

      )})

    </>

  )}

</div>

);
```

Output:

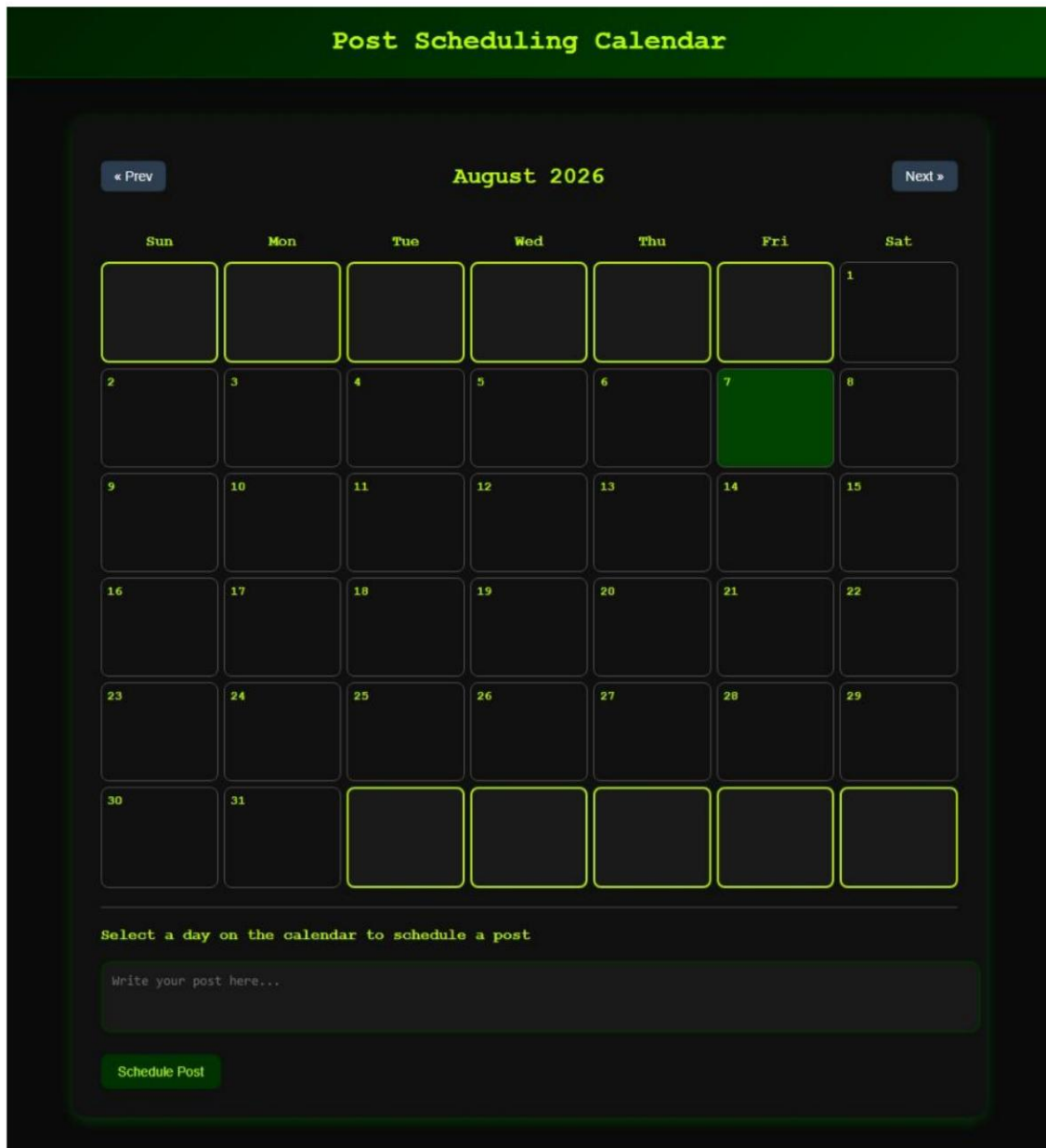


Fig A: Output Calendar

Learning Outcomes:

- Understand the principles of time-based data visualization.
- Design and implement an interactive calendar interface.
- Integrate structured data with calendar components.
- Implement drag-and-drop interactions for event management.
- Build responsive and user-friendly scheduling applications using modern web technologies.
- Gain practical experience in React component integration and state management.

Conclusion:

The interactive calendar interface provides an efficient and intuitive solution for scheduling and managing posts. By combining calendar visualization with interactive features such as drag-and-drop and event editing, users can organize content more effectively.

EXPERIMENT – 4.2

Aim:

To optimize rendering performance and implement testing strategies for interactive UI components, ensuring efficient rendering, improved responsiveness, and reliable application behaviour.

Objectives:

- To understand performance bottlenecks in UI systems.
- To optimize rendering using memorization techniques.
- To reduce unnecessary re-renders.
- To implement testing for UI components and application logic.
- To improve application reliability through automated testing.

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Frontend: React.js
- Backend: Node.js (optional)
- Code Editor: Visual Studio Code
- Package Manager: npm
- Testing Framework: Jest
- Testing Library: React Testing Library
- Browser: Google Chrome / Microsoft Edge

Theory:

Performance optimization is an essential aspect of modern web application development. As applications become more interactive and data-intensive, unnecessary component re-rendering can negatively affect user experience. React provides optimization techniques such as `React.memo`, `useMemo`, and `useCallback` to minimize redundant rendering and improve application efficiency.

Testing ensures that UI components and application logic function correctly under different scenarios. Automated testing using Jest and React Testing Library helps developers verify component rendering, user interactions, and business logic while reducing the chances of bugs during development and maintenance.

Implementation:

1. Create a React application and identify components with frequent re-renders.
2. Analyse rendering performance using React Developer Tools.
3. Apply `React.memo` to prevent unnecessary component rendering.

4. Use `useMemo` to memorize expensive computations.
5. Use `useCallback` to memorize callback functions passed as props.
6. Write unit tests for UI components using Jest and React Testing Library.
7. Test user interactions such as button clicks, form inputs, and state updates.
8. Validate application logic and component behavior.
9. Measure performance improvements after optimization.

Code:

- **App.js**

```
import { useState } from "react";

const WEEKDAYS = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
const MONTH_NAMES = [ "January", "February", "March", "April", "May", "June",
  "July", "August", "September", "October", "November", "December"];

function getMonthGrid(year, month) {
  const firstDay = new Date(year, month, 1).getDay();
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const cells = [];
  for (let i = 0; i < firstDay; i++) cells.push(null);
  for (let d = 1; d <= daysInMonth; d++) cells.push(d);
  while (cells.length % 7 !== 0) cells.push(null);

  return cells;
}

function dateKey(year, month, day) {
  return `${year}-${month + 1}-${day}`;
}

function App() {
  const today = new Date();
  const [year, setYear] = useState(today.getFullYear());
  const [month, setMonth] = useState(today.getMonth());
  const [firstDay, setFirstDay] = useState(0);
```

```

    cursor: day ? "pointer" : "default"

    }}

  >

  {day && (

    <>

    <div style={{ fontWeight: "bold", fontSize: "13px" }}>{day}</div>

    {dayPosts.map((post, i) => (

      <div>

        key={i}

        draggable

        onDragStart={() => handleDragStart(key, i)}

        className="coder-code-area"

        style={{

          backgroundColor: "#003300", color: "#c0ff00",

          fontSize: "11px", borderRadius: "4px", padding: "3px 5px",

          marginTop: "4px", cursor: "grab", display: "flex",

          justifyContent: "space-between",

          alignItems: "center"

        }}

        <span style={{ overflow: "hidden", textOverflow: "ellipsis" }}>{post}</span>

        <span onClick={(e) => { e.stopPropagation(); deletePost(key, i); }} style={{

marginLeft: "4px", cursor: "pointer" }}>X</span>

      </div>

    )}}

  </>

  )}

</div>

);

}}}

export default App;

```

- **App.test.js**

```
import { render, screen, fireEvent } from '@testing-library/react';
import App from './App';
import { MONTH_NAMES } from './utils/dateUtils';

function expectedCurrentLabel() {
  const now = new Date();
  return `${MONTH_NAMES[now.getMonth()]} ${now.getFullYear()}`;
}

test('navigating to the next month advances the label', () => {
  render(<App />);
  const now = new Date();
  fireEvent.click(screen.getByLabelText('next-month'));
  const nextMonth = (now.getMonth() + 1) % 12;
  const nextYear = now.getMonth() === 11 ? now.getFullYear() + 1 : now.getFullYear();
  expect(screen.getByTestId('header-label')).toHaveTextContent(
    `${MONTH_NAMES[nextMonth]} ${nextYear}`
  );
});

test('navigating to the previous month rewinds the label', () => {
  render(<App />);
  const now = new Date();
  fireEvent.click(screen.getByLabelText('previous-month'));

  const prevMonth = now.getMonth() === 0 ? 11 : now.getMonth() - 1;
  const prevYear = now.getMonth() === 0 ? now.getFullYear() - 1 : now.getFullYear();
  expect(screen.getByTestId('header-label')).toHaveTextContent(
    `${MONTH_NAMES[prevMonth]} ${prevYear}`
  );
});
```

- **App.css**

```
.App {
  text-align: center;
}

.App-logo {
  height: 40vmin;
  pointer-events: none;
}

@media (prefers-reduced-motion: no-preference) {
  .App-logo {
    animation: App-logo-spin infinite 20s linear;
  }
}

.App-header {
  background-color: #282c34;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  font-size: calc(10px + 2vmin);
  color: white;
}

@keyframes App-logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
```

Output:

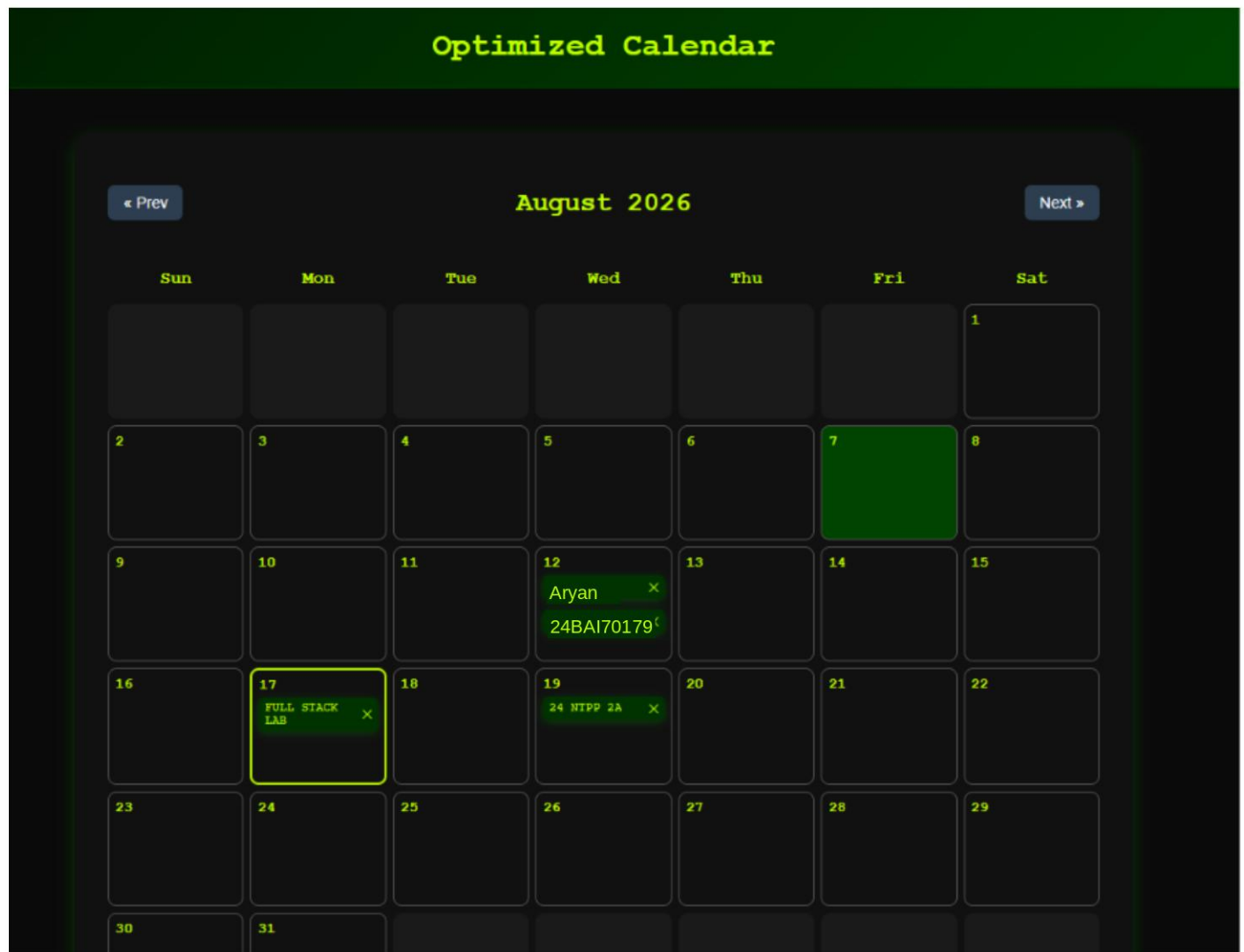


Fig A: Optimized Calendar

Learning Outcomes:

- Understand the causes of unnecessary component re-rendering.
- Apply memoization techniques to optimize React applications.
- Improve rendering performance and application responsiveness.
- Write effective unit tests for UI components.
- Verify application behavior using automated testing tools.
- Develop efficient, maintainable, and reliable React applications.

Conclusion:

Performance optimization and testing are fundamental practices in modern frontend development. By implementing memorization techniques and automated testing, developers can significantly improve application efficiency, reduce rendering overhead, and ensure reliable user experiences. This experiment provides practical knowledge of building high-performance and well-tested React applications.